Reliability Engineering and System Safety 272 (2026) 112588

Contents lists available at ScienceDirect

Reliability Engineering and System Safety

journal homepage: www.elsevier.com/locate/ress

A deep-learning-based approach for temporal transportation network dismantling

Li Hong [a], Yifan Zhu [b], Kaishuo Liu [b], Yu Liu [a], Changjun Fan [c], Mengqiao Xu [b,*]

[a] School of Software Technology, Dalian University of Technology, No. 321 Tuqiang Street, Economic and Technological Development Zone, Dalian City, Liaoning Province, 116620, China
[b] School of Economics and Management, Dalian University of Technology, No.2 Linggong Road, Ganjingzi District, Dalian City, Liaoning Province, 116024, China
[c] College of Systems Engineering, National University of Defense Technology, Changsha Province, 410073, China

ARTICLE INFO

Keywords:
Temporal transportation networks
Temporal network dismantling problem
Directed acyclic graph
Multi-head graph attention networks

ABSTRACT

Accurately assessing the importance of individual nodes at different time for maintaining the efficiency of temporal transportation networks is an important challenging problem. It is for the first time that this issue has been formulated as a temporal network dismantling (ND) problem, which aims to identify the minimal set of nodes with time information whose disruption can rapidly degrade temporal network efficiency. This paper proposes a deep-learning-based network dismantling framework for its solution. Specifically, a directed acyclic graph representation is used to capture the continuous topological changes in a temporal network. Then, our framework employs multi-head graph attention networks to learn latent higher-order features by aggregating different temporal features of nodes and their neighbors, and incorporates the shortcut connections technique to mitigate the vanishing gradient problem. Extensive experiments on synthetic and real-world networks demonstrate that the proposed approach outperforms baselines and exhibits significant robustness. Furthermore, an early warning signal indicator is adopted to quantify the dismantling risk of temporal networks. Our approach offers a practical tool for identifying critical nodes in temporal transportation networks, thereby assisting stakeholders in enhancing robustness of transportation services.

1. Introduction

Most real-world transportation systems, including global liner shipping networks [1], railway networks [2], subway networks [3], and airline networks [4], typically operate according to predetermined schedules. These scheduled transportation networks are represented as temporal networks, which not only exhibit complex topological structures but also demonstrate significant temporal dependencies [5]. The operational functionality of temporal transportation networks is significantly vulnerable when facing undesirable events (e.g., natural disasters, equipment failures, or accidents). Accurately assessing the significance of individual nodes at different time for maintaining network performance is a crucial issue. Such temporal networks are largely influenced by a small number of nodes with time information, whose failure or deliberate attacks may significantly degrade certain network functionality [6–10]. This finding motivates us to focus on identifying the minimal set of nodes with time information whose disruption can quickly dismantle network performance, which is modeled as the temporal network dismantling (ND) problem. Effectively addressing such temporal ND problem is an important and challenging task in network science.

Considering the time-varying characteristics of temporal transportation network structures, it is crucial to first construct a reasonable graph representation for temporal transportation networks. Some existing studies on graph representations of temporal networks [2,7,11–19] aggregate temporal information into a static graph or discretize it into consecutive time-based snapshots. However, such methods ignore the temporal information of connection paths between nodes and fail to fully capture the continuous evolution of network topology over time. Therefore, they have difficulty in accurately characterizing the performance of temporal networks and quantifying the network's performance degradation under node failures occurring at different time. To address this issue, some studies [20–23] have represented temporal networks as temporal graphs. Although temporal graph modeling methods can effectively preserve the temporal information in networks, such methods remain in the exploratory stages. Inspired by studies [22] and

* Corresponding author.
E-mail address: stephanie@dlut.edu.cn (M. Xu).

https://doi.org/10.1016/j.ress.2026.112588
Received 6 November 2025; Received in revised form 27 January 2026; Accepted 11 March 2026
Available online 12 March 2026

[23], a directed acyclic graph composed of nodes and unidirectional edges is obtained by transforming the original temporal graph. Its acyclic nature ensures that no node can be revisited through a directed path, thereby naturally avoiding circular dependencies and reducing redundant paths. A directed acyclic graph temporal network model enables a precise representation of time-respecting paths [24] between nodes, which are described as sequences of time-stamped edges with monotonically increasing timestamps. Furthermore, it provides a foundational framework for quantifying network performance.

The traditional static network dismantling problem focuses on finding the minimal set of nodes whose removal rapidly dismantles a network [25–28]. The network dismantling problem belongs to the discrete combinatorial optimization problem that exhibits NP-hard nature [29]. In temporal networks, the topological structure evolves over time, and the impact of removing the same node at different time on network performance may vary significantly. Most existing studies have adopted centrality-based methods to quantify the significance of each node at different time [2,13–15,18,20]. Although individual basic temporal centrality methods typically capture either global or local features of a temporal network, incorporating different temporal centrality metrics enables both to be captured simultaneously. Recently, some studies have proposed machine-learning-based methods for tackling the static network dismantling problem, leading to significant breakthroughs in this area of research [30–38]. These machine learning approaches have shown strong effectiveness in handling the static network dismantling problem. Hence, it is promising to leverage machine-learning-based approaches for addressing the temporal network dismantling problem.

Therefore, to accurately assess the importance of individual nodes at different time for maintaining the efficiency of temporal transportation networks, this paper formulates a mathematical model of temporal ND problem and then proposes a deep-learning-based dismantling method to solve it. Specifically, the contributions of our methodology are twofold. First, to the best of our knowledge, it is for the first time that the issue of accurately assessing the importance of each node at different time for maintaining network performance has been formulated as a temporal ND problem. The temporal ND problem is defined as identifying the minimal set of nodes with time information whose disruption can rapidly degrade network performance. In formulating the temporal ND problem model, we adopt a directed acyclic graph representation, which captures the continuous topological changes in a temporal transportation network and provides a foundational framework for quantifying network performance. Second, our proposed deep-learning-based dismantling method proves superior performance over centrality-based methods in solving such temporal ND problem. Specifically, it generates the higher-order features through the aggregation of each node's and its neighbors' temporal features, which are then used to compute the dismantling score for each node at different time. Our method employs multi-head graph attention networks (GAT) to capture latent higher-order temporal features and incorporates the shortcut connections technique (SC) to mitigate the vanishing gradient issue, thereby improving the representational power and learning efficiency of neural networks. Our model is trained under supervised learning on small-scale synthetic instances but demonstrates strong generalization capabilities across various large-scale synthetic networks and real-world temporal transportation networks. Finally, inspired by the work of [31], the dismantling scores calculated by our approach are applied to predict the impact of future failures or attacks on transportation network performance, by deriving an early warning signal indicator to quantify network dismantling risk.

This paper is structured into the following sections. Section 2 surveys existing related studies. Section 3 provides our methodology, which includes the formulation of a mathematical model for the temporal ND problem and the development of a deep-learning-based method to solve it. Section 4 reports the primary experiments and results analysis. Finally, Section 5 presents the study's conclusion, highlighting the main findings and their implications.

2. Related works

Related studies on graph representation of temporal networks and methods for dismantling temporal networks are reviewed in this section. The temporal ND problem is a discrete combinatorial optimization problem. Current research on this issue remains scarce, and a few existing methods primarily adopt centrality-based methods.

2.1. Graph representation of temporal networks

The graph representation of temporal networks serves as the foundation for modeling and understanding the dynamic behavior of systems [5]. This section reviews the primary modeling approaches of temporal networks and the progress in related research. Generally, temporal networks mainly have three categories of graph representations.

The first category of temporal network modeling is the aggregated static graph [16,17], in which the system's dynamics are aggregated into a weighted static structure. The aggregated static graph compresses temporal information into the edge weights or node attributes of a static graph, simplifying the process of extracting temporal information [16]. The study [39] aggregates the connecting edges in the static graph structure as directed edges to better analyze the information diffusion mechanisms in temporal social networks, revealing which nodes or paths are more important in a network. A key benefit of the aggregated static graph representation lies in its simple modeling, direct application of powerful static graph algorithms, and high computational efficiency. However, this type of graph representation ignores detailed temporal information such as the specific time of each connection, failing to capture the temporal dependencies of interactions. Thus, its application is limited to finding critical nodes in temporal networks.

The second category of temporal network modeling relies on temporal snapshots [2,7,12,18,19,40–44], which are widely used for analyzing the dynamic evolution of networks, changes in node influence, and mechanisms of information propagation. Discretized into a series of sequential snapshots across different time, the temporal network is represented such that all of these snapshots are modeled as static graphs. For instance, the study [2] proposes a spatiotemporal train service network, which represents the railway transportation service based on train timetables. The study [44] constructs annual global maritime networks from 1977 to 2008 using years as time slices, with nodes representing ports and edges representing shipping connections. Since this type of graph representation can reflect the temporal evolution of networks to some extent through sequential snapshots, it is widely used to measure node centrality in temporal networks [2,12,13,19,40–42]. However, such network representation methods based on temporal snapshots remain essentially static and ignore the connections between nodes across different time slices, making them unable to fully capture the continuous topological evolution of a temporal network.

The third category of temporal network modeling is temporal graphs [20–23,45], which can more accurately model the evolution and behavior of real-world systems over time. This representation structure based on temporal graphs can effectively preserve the temporal information in networks. However, related research is still in the exploratory stage. For instance, the study [20] uses a temporal graph method to construct a small-scale regional maritime temporal network and proposes a novel temporal centrality measure to evaluate the significance of ports. However, the temporal graph representation method in the study [20] has limitations, as the network scale expands dramatically due to the introduction of the time dimension, making it difficult to scale effectively to the construction of large-scale temporal networks such as global container liner networks and the computation of temporal paths. Inspired by studies [22] and [23], we transform a temporal graph into a directed acyclic graph composed of nodes and unidirectional edges. Its acyclic nature ensures that no node can be revisited through a directed

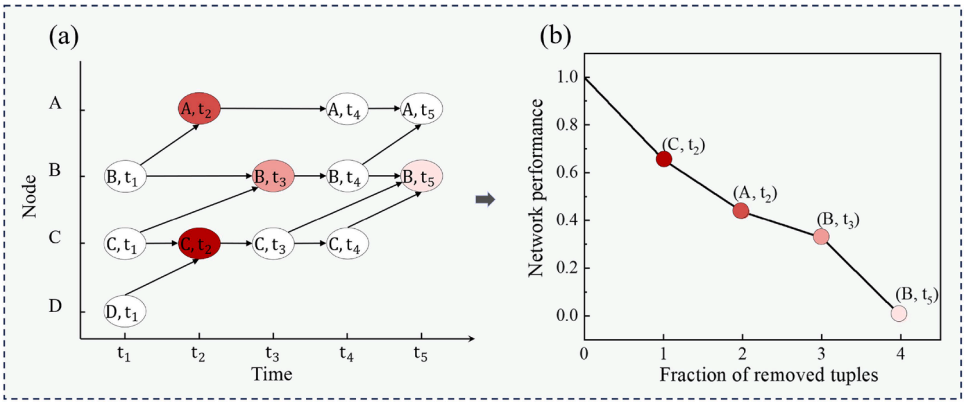

Fig. 1. Illustration of the dismantling of a toy temporal transportation network. (a) A graph representation of a temporal transportation network, where the horizontal axis denotes time and the vertical axis denotes nodes; (b) The performance degradation process of the temporal transportation network under the sequential removal of tuples, where the horizontal axis shows the number of removed tuples and the vertical axis shows network performance. Note: a (node, time) tuple with a darker color indicates greater importance for maintaining network performance.

path, thereby naturally avoiding circular dependencies and reducing redundant paths. By explicitly representing temporal dependencies (e.g., edges indicating the chronological order of events), directed acyclic graphs enable efficient and accurate modeling of the inherent dynamic topology in temporal graphs.

Based on the perspective of temporal graphs, this paper adopts a directed acyclic graph temporal network modeling method to fully capture the continuous topological evolution of temporal transportation networks, thereby achieving accurate characterization of temporal paths between nodes. Furthermore, this modeling method provides a fundamental framework for quantifying network performance.

2.2. Methods for dismantling temporal networks

Dismantling temporal networks is a critical problem with applications in epidemic control, infrastructure resilience, and social network analysis [6,7,12]. Unlike static networks, temporal networks introduce the time dimension, which makes addressing the temporal network dismantling problem more challenging.

Existing studies have adopted centrality-based methods for identifying the critical nodes with time information that most significantly impact the performance of temporal networks [2,13–15,18,20,46]. For instance, the studies [2] and [18] propose two improved k-shell methods to assess node criticality over time in temporal networks. The study [20] proposes a new temporal node centrality metric called “Spread Potential”, which evaluates the significance of each node at different time in temporal networks. The study [13] proposes a time-ordered graph model and extends traditional topological measures (e.g., degree, betweenness, and closeness centrality) to dynamic network scenarios. The studies [14] and [15] respectively propose improved temporal betweenness centrality measures to evaluate critical nodes affecting the performance of temporal networks. The study [46] presents a new sequential-path tree-based centrality (SPT-C) method to detect critical nodes in temporal networks. The above methods are effective for assessing the significance and influence of nodes within temporal networks. Although individual basic temporal centrality methods typically capture either global or local features of a temporal network, incorporating different temporal centrality metrics enables both to be captured simultaneously.

Recently, some studies have proposed machine-learning-based methods for tackling the static network dismantling problem, leading to significant breakthroughs [30–36,47,48]. Promising results have been achieved by these machine learning approaches in addressing the static network dismantling problem. For instance, the studies [31,32] and [47] have designed deep neural models that can learn higher-order features through the aggregation of node and neighborhood information to tackle the static network dismantling problem. These models are trained using supervised learning methods. Motivated by these studies, we aim to explore machine-learning-based approaches that incorporate different temporal centrality features for addressing the temporal ND problem of transportation networks.

3. Methodology

In this section, we first solve the temporal network dismantling problem by formulating a mathematical model. Then, we propose a deep-learning-based dismantling framework to solve this problem.

3.1. Problem formalization

3.1.1. Description of temporal network dismantling problem

Formally, we represent a temporal transportation network as a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of nodes and $\mathcal{E}$ denotes the set of temporal edges. A temporal edge $e \in \mathcal{E}$ is defined as a quadruple $e = (u, v, t_u, \lambda)$, where u and v ($u, v \in \mathcal{V}$) respectively denote the starting node and arrival node, t_u indicates the starting time, λ represents the travel time from u to v starting at time t_u, and $t_u + \lambda$ indicates the arrival time. Following previous studies [22] and [23], we transform a temporal graph into a directed acyclic graph composed of nodes and unidirectional edges. Its acyclic nature ensures that no node can be revisited through a directed path, thereby naturally avoiding circular dependencies and reducing redundant paths. Specifically, we present the transformation of a temporal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ into a corresponding directed acyclic graph representation, denoted as $G = (V, E)$. The detailed construction of G involves two parts:

(1) We construct the set of nodes, denoted as V. For each node $v \in \mathcal{V}$, create the nodes in V as follows. First, denote $\Pi(u, v)$ and $\Pi(v, w)$ respectively as the sets of temporal edges from u to v and from v to w, where $w \in \mathcal{V}$ is the arrival node. t_v indicates the starting time. We define the time set $T(v)$ of v as $T(v) = \{t_u + \lambda : (u, v, t_u, \lambda) \in \Pi(u, v)\} \cup \{t_v : (v, w, t_v, \lambda) \in \Pi(v, w)\}$, that is, $T(v)$ contains all the arrival time at v and the starting time from v. Then, $|T(v)|$ copies of node v are created, and each node in V is expressed as a (node, time) tuple, i.e., (v, t), where t is the arrival time or starting time in $T(v)$. We define the node set V as $V = \{(v, t) : t \in T(v)\}$. Finally, the nodes in V are sorted in descending temporal order, such that for any $(v, t_1), (v, t_2) \in V$, if $t_1 > t_2$, then (v, t_1) is placed before (v, t_2).

(2) We construct the set of edges, denoted as E. Create the edges in E as follows. First, assume V is given by $\{(v, t_1), (v, t_2), \ldots, (v, t_x)\}$, where $t_{i+1} > t_i$ and $1 \leq i < x$. Construct a directed edge from

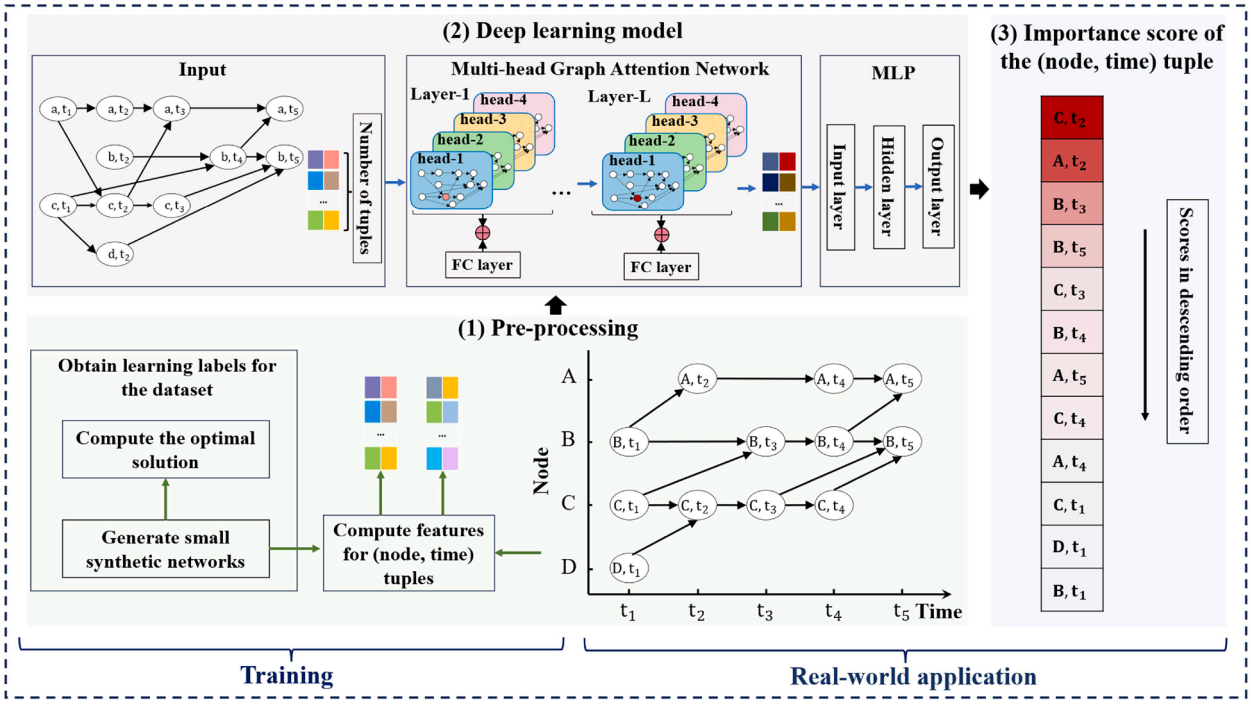

Fig. 2. A deep-learning-based dismantling framework for the temporal ND problem.

(v, t_i) to (v, t_{i+1}) for $1 \le i < x$. If $x \le 1$, no edge is constructed. Second, for each temporal edge $e = (u, v, t_u, \lambda) \in \mathscr{E}$, construct a directed edge from (u, t_u) to $(v, t_u + \lambda)$.

The description of the temporal ND problem is as follows: Given a directed acyclic graph representation G of a temporal transportation network within a time window, where each node is represented as a (node, time) tuple, the objective is to identify an optimal sequence for removing these tuples one by one so as to maximize the degradation of network performance. If node v is disrupted at time t, then when calculating network performance, we remove from graph G the temporal edges arriving at v from any other node u at t and the temporal edges starting from v to any other node w at t. Consider a toy temporal transportation network as an example, consisting of 4 nodes (i.e., A, B, C, D) and temporal edges. Fig. 1(a) shows the directed acyclic graph representation of the temporal transportation network, which contains 12 (node, time) tuples. Under the optimal removal sequence of these tuples, network performance is completely dismantled after only 4 tuples are removed (Fig. 1(b)).

3.1.2. Formulation of mathematical model

Network performance is represented by network efficiency, which is defined as the average of the reciprocals of the temporal shortest path lengths across all pairs of tuples with distinct nodes, as shown in Eq. (1). This definition extends the classical measure of network efficiency in static networks to temporal networks by explicitly incorporating time-respecting paths and temporal ordering. Specifically, given a starting node u at time t_u and an arrival node v ($u \ne v$), we define the temporal shortest path as the earliest-arrival path [23], i.e., the time-respecting path that yields earliest arrival time starting from u to v given a starting time t_u. The corresponding temporal shortest path length is measured as the time difference between arrival time and starting time along the earliest-arrival path. For many real-world transportation systems, reaching a destination as early as possible is a primary performance objective. Therefore, defining the temporal shortest paths based on the earliest arrival paths provides a more realistic and operationally meaningful measure of network efficiency. In the directed acyclic graph representation, each node is represented as a (node, time) tuple, and edges represent time-respecting interactions. By construction, all edges point from earlier time stamps to later ones, which guarantees that the resulting graph is acyclic. This property enforces the chronological constraint of temporal paths, and thus guarantees the accuracy of the temporal shortest-path computation. Dijkstra's algorithm is employed to accurately compute the temporal shortest paths in a directed acyclic graph. The directed acyclic graph representation significantly improves computational efficiency. Since no cycles exist, the temporal shortest paths can be computed without repeated relaxations caused by time loops.

$$E' = \frac{1}{m}\sum_{u \ne v}\frac{1}{dis\_t_{uv}} \tag{1}$$

where E' represents network efficiency; m denotes the total number of (node, time) tuple pairs from any node u to any other node v ($u \ne v$). dis_t_{uv} denotes the temporal shortest path length from u to v given a starting time t_u in a directed acyclic graph. Note: dis_t_{uv} is set to ∞ when no temporal path exists from u to v at t_u. Unlike the shortest path in a static network, the temporal shortest path must follow the chronological order in which connections between nodes occur within the time window.

The mathematical model of the temporal network dismantling problem is formulated as follows: Given a directed acyclic graph representation $G = (V, E)$ of a temporal transportation network within a time window. The objective is to determine the optimal removal sequence of the (node, time) tuples such that the network efficiency degrades most rapidly as the number of removed tuples increases—that is, to minimize the following Accumulated Normalized Efficiency (ANE) [30,49], as defined in Eq. (2). A smaller ANE value implies a higher effectiveness of the dismantling method.

$$\text{Min ANE} = \frac{1}{N}\sum_{n=1}^{N}\frac{E(n)}{E} \tag{2}$$

where $E(n)$ represents network efficiency after removing n (node, time) tuples, and E denotes the initial network efficiency before any (node,

time) tuples are removed. N denotes the number of (node, time) tuples in the initial network.

3.2. A deep-learning-based dismantling framework

A deep-learning-based dismantling framework is developed to address the temporal ND problem. Basic temporal centrality measures [13], such as temporal degree centrality and temporal betweenness centrality, are effective methods for characterizing the significance and influence of nodes in a temporal network. Although individual basic temporal centrality methods can capture either global or local features of a temporal network, incorporating different temporal centrality metrics enables both to be captured simultaneously. Therefore, building upon individual basic temporal centrality methods, we further explore machine learning approaches that incorporate different temporal centrality metrics, aiming to rapidly dismantle network efficiency.

Specifically, we design a deep-learning-based framework for dismantling temporal transportation networks. By identifying and removing a small number of the most critical tuples, our framework aims to rapidly disintegrate network efficiency. Our framework primarily employs the multi-head graph attention networks (GAT) [50] and incorporates the shortcut connections technique (SC) [51,52]. The proposed model aggregates the temporal degree centrality and temporal betweenness centrality features of (node, time) tuples and their neighbors to generate higher-order feature representations, aiming to capture the complex structures and interaction patterns in a temporal network. Inspired by the work of [52], we integrate the SC technique to mitigate the vanishing gradient problem and improve neural networks' learning capacity. Furthermore, the proposed model is trained on small synthetic networks and then generalized to large real-world temporal transportation networks, achieving efficient evaluation across different scales. As illustrated in Fig. 2, our framework is structured into three primary modules.

(1) Pre-processing. Pre-processing (Fig. 2(1)) mainly involves generating synthetic small-scale temporal networks as the training dataset and computing the corresponding learning labels and basic temporal features. First, considering the scale-free characteristic of many real-world networks, this study adopts the classic Barabási-Albert (BA) [53] model to generate small synthetic temporal networks as a training dataset. The size of each small synthetic network is randomly selected from the range 15 to 20. Second, to obtain the labeled data required for supervised learning, a brute-force search method is used to identify the optimal disintegration sets for each synthetic network. Specifically, this study identifies all the optimal solutions via a brute-force method so that the temporal network efficiency is degraded to a predefined target (e.g., 10 % of the original network efficiency in our experiments). Then, each tuple's label is calculated by dividing the number of times it appears in the optimal solution set by the total number of optimal solutions. Thus, each (node, time) tuple is assigned a numerical label, denoted as *label_score*, where the value depends on whether the tuple is included in the optimal dismantling set. For instance, if there is only one optimal dismantling set, we assign the value 1 to the tuples in that set and 0 to the remaining tuples; if there are two optimal dismantling sets, we assign the value 1 to the tuples present in both sets, 0.5 to those present in only one set, and 0 to all other tuples. The *label_score* is the learning objective for our model—teaching it that certain tuples exhibit heightened criticality based on their occurrence in multiple optimal dismantling sets. Finally, we calculate the basic temporal features of each (node, time) tuple (e.g., temporal degree centrality (TDC) as shown in Eq. (3) and temporal betweenness centrality (TBC) as shown in Eq. (4)) to preliminarily characterize each tuple's importance in a network. The calculation of TDC is as follows. For a directed acyclic graph $G = (V, E)$ of a temporal network within a given time window $[i, j]$, the temporal degree centrality of a node $v \in V$, represented by $D_{ij}(v)$, is defined as:

$$D_{ij}(v) = \sum_{t=i}^{j} \left(d_t^{in}(v) + d_t^{out}(v) \right) \tag{3}$$

where $d_t^{in}(v)$ represents the in-degree of node v (i.e., the number of inbound edges to v); $d_t^{out}(v)$ represents the out-degree of node v (i.e., the number of outbound edges from v). Note that all self-loop edges from v_{t-1} to v_t are disregarded for all $t \in \{i+1, \ldots j\}$. The calculation of TBC is as follows. For a directed acyclic graph $G = (V, E)$ of a temporal network within a given time window $[i, j]$, the temporal betweenness centrality of a node $v \in V$, represented by $B_{ij}(v)$, is defined as the proportion of all the temporal shortest paths passing through node v during the time interval $[i, j]$ among the total number of temporal shortest paths between all pairs of nodes. The $B_{ij}(v)$ is given by:

$$B_{ij}(v) = \sum_{t=i}^{j} \sum_{\substack{m \neq v \neq n \in V \\ \varphi_{tj}(m,n) > 0}} \frac{\varphi_{tj}(m, n, v)}{\varphi_{tj}(m, n)} \tag{4}$$

where $\varphi_{tj}(m, n)$ denotes the total number of temporal shortest paths from node m to node n in G, and $\varphi_{tj}(m, n, v)$ denotes the number of paths from node m to node n that pass-through node v. These temporal centrality metrics are then normalized and concatenated to form the initial temporal feature vector f of each tuple, i.e., $f = \left[D_{ij}(v), B_{ij}(v) \right]$ and $v \in V$. We use the low-dimensional continuous vector f as the initial input features for the deep learning models.

(2) Training a deep learning model. To capture latent higher-order features in a temporal network, we design a deep learning model (Fig. 2(2)) built upon the multi-head graph attention network (GAT) [50]. Meanwhile, our model uses fully connected (FC) layers as the shortcut connections technique (SC), which facilitates the propagation of gradients and mitigates the vanishing gradient issue, thereby enhancing the model's stability and representation capability [52]. Specifically, we input the initial temporal feature vector f of the (node, time) tuple and the network topology into our model. The SC is implemented between consecutive layers in the deep learning architecture by directly adding the previous layer's output to the subsequent layer's output. This introduces an identity mapping into the network, allowing the model to focus on learning the residual rather than relying entirely on the nonlinear transformation from scratch. To maintain dimensional consistency, FC layers are employed within the shortcut path to adjust the input feature dimension so that it matches the dimension of the nonlinearly transformed output. Through multiple rounds of neighbor tuple features weighted aggregation and iterative updating of the embedding vectors, the model generates the higher-order features embedding representation $Emb(f)$ that captures the complex correlations of the tuple. $Emb(f)$ is defined by Eq. (5). Our deep learning model consists of the L-layer GAT network integrated with the shortcut connections technique (SC). The model generates a dismantling score for each tuple v by considering the L-hop neighborhood of the tuple. We assume a tree with tuple v as the root and height L, where each tuple's neighbors are its child tuples. In other words, the $(l+1)$-th level of the tree consists of the neighbors of the tuples in the l-th level. For example, the first level corresponds to the direct neighbors of tuple v. The high-order features $Emb(f)$ of tuple v are computed in a bottom-up manner by aggregating the information from the L-hop neighborhood. Each layer of the GAT network processes one layer of information from the tree, so as the model deepens, the neighborhood from which the information is drawn becomes increasingly distant. Specifically, the model starts from the bottom layer of the tree,

progressively calculates the high-order features of each tuple at the L-th layer, and transmits the information upwards until it reaches the root tuple v. This means the model can aggregate the information from the entire L-hop neighborhood of tuple v into $Emb(f)$. Thanks to the self-attention mechanism in GAT, this process can also account for the varying importance of different tuples within the neighborhood. Additionally, we introduce the SC to mitigate the vanishing gradient problem. Notably, we accounted for directional information when computing the labels using the brute-force search method. Since the dismantling effect in directed networks highly depends on the direction of edges, these labels essentially capture the directional dependencies in the network, such as path reachability and the importance of in-degree or out-degree. Powerful supervisory signals, namely the labels of the optimal dismantling set, are expected to drive the model to learn the latent structural patterns in directed networks as much as possible. Specifically, since the labels for the temporal ND task are computed based on a directed acyclic graph, the model will gradually learn to identify and fit direction-dependent information through statistical regularities in the training data. For instance, tuples with high in-degree are generally more critical, as their removal affects numerous downstream tuples that depend on their information, or attacking tuples located on critical paths can significantly disrupt network efficiency.

$$Emb(f) = \sigma\left(Fun\left(f,\ w_g\right) + f \cdot w_r + b_e\right) \tag{5}$$

where σ is the activation function, and the ELU activation function is used in our study; the nonlinear function $Fun(f, w_g)$ represents the output obtained after applying the concatenation and dropout layers of the multi-head attention mechanism; $f \cdot w_r$ denotes the results of shortcut connections technique; w_g, w_r, and b_e are trainable parameters in our model.

Then, the encoded higher-order feature information is fed into a Multilayer Perceptron (MLP) [54]. Through multiple layers of nonlinear transformations, the feature vector is mapped to the [0,1] interval as probabilities. Our model outputs the normalized probability value according to Eq. (6), which represents the dismantling score of the tuple.

$$dis\_score = \sigma(Emb(f) \cdot w_s + b_s) \tag{6}$$

where w_s denotes the trainable parameter; b_s denotes the trainable bias vector; σ denotes the activation function and this study uses the ELU activation function; *dis_score* represents the tuples' dismantling score.

The loss function of our model: For tuples in the training dataset x ($x \in X$), the scores are denoted as *label_score*, while *dis_score* represents the predicted scores by our model. We adopt the Mean Squared Error (MSE) as the loss function for model training, as shown in Eq. (7). Our model is implemented utilizing the PyTorch framework and trained with the Adam optimizer.

$$Loss = \frac{1}{|X|} \sum_{x \in X} \left(label\_score_x - dis\_score_x\right)^2 \tag{7}$$

where X represents the entire training dataset.

(3) Application. Our pre-trained model outputs the predicted importance score of the (node, time) tuple in real-world networks. Specifically, input the topology of a real-world temporal transportation network and the initial features of (node, time) tuples (e.g., TDC as shown in Eq. (3) and TBC as shown in Eq. (4)) into a pre-trained model. Then, through the forward inference process, the model generates an importance score for each tuple. The score is in the range [0,1], with higher values indicating greater importance of a tuple in maintaining the network efficiency. Therefore, the tuples' dismantling scores are obtained, allowing for visual interpretation where darker color signifies greater importance of the (node, time) tuple, as shown in Fig. 2(3).

4. Experiments and results

We analyze the performance of our approach across synthetic networks and real-world transportation networks. This study uses degree-heterogeneous Barabási-Albert (BA) networks [53] and degree-homogeneous Watts-Strogatz (WS) networks [55] as synthetic networks to verify the effectiveness of our approach. The real-world transportation systems include high-speed rail networks, global liner shipping networks, airline schedule networks, subway schedule networks, etc. The following describes our model configuration and hyperparameter settings: the maximum training epochs are 300; the model uses 4 self-attention heads and 4 GAT layers; the number of MLP layers is 4; the batch size is 32; the neurons per layer are set to 64, 32, 16, and 8; The node embeddings are 64-dimensional for all layers. Notably, the learning rate of Adam optimizer, the learning rate decay and the probability of dropout are identified as the primary optimization-related hyperparameters of our model. To assess the impact of hyperparameter settings on model performance, we perform a sensitivity analysis by examining the evolution of the training loss during the optimization process, as illustrated in Fig. A.1 of Appendix A. Multiple experimental trials are conducted with different hyperparameter settings to identify the optimal configuration. The model achieves optimal performance when the learning rate decay, the learning rate, and the dropout probability are set to 10^{-5}, 10^{-4}, and 0.2, respectively, as indicated by the red curve. All computations were performed utilizing a system featuring an Intel i7–12,700 processor operating at 2.10 GHz. Furthermore, inspired by the work of [31], an early warning signal indicator is applied to quantify network dismantling risk.

4.1. Effectiveness comparison on synthetic networks

The majority of real-world transportation systems discussed here display structural heterogeneity akin to the BA model. Therefore, the BA model is employed in our framework during the training phase. Specifically, we utilize the BA model to construct 600 training temporal networks, with the size of each small synthetic network randomly selected from the range 15 to 20. This study utilizes a supervised learning approach for training our model. While trained on small, optimally solvable synthetic networks, our model generalizes effectively to large networks where optimal solutions are computationally intractable.

Our approach (named Ours) is compared against baseline methods: (1) Temporal Degree Centrality (TDC) [13], which removes tuples according to their temporal degree centrality in descending order. (2) Temporal Betweenness Centrality (TBC) [13], which removes tuples according to their temporal betweenness centrality in descending order. (3) Degree Centrality (DC) [56], which removes tuples based on decreasing degree. A higher tuple degree indicates a higher degree centrality, which reflects the tuple's greater importance in a network. (4) Betweenness Centrality (BC) [57], which is a metric that quantifies a tuple's significance based on how often it appears in shortest paths. (5) Label: Each synthetic network in the training dataset undergoes an optimal dismantling process via a brute-force search method to obtain the learning target. (6) Directed Node Entanglement (DNE) [58], which is a recent study on dismantling directed networks using a multi-temporal information field approach. Due to the high computational complexity of the DNE method, we only apply it to solve small-scale networks. (7) Collective Influence (CI) [59] is a popular method for identifying influential nodes in a network by considering the collective effect of neighboring nodes. (8) CoreHD [60] is a classic heuristic network dismantling method that iteratively removes the highest-degree nodes from the k-core of the network. (9) Graph Dismantling with Machine learning (GDM) [31], which is an effective

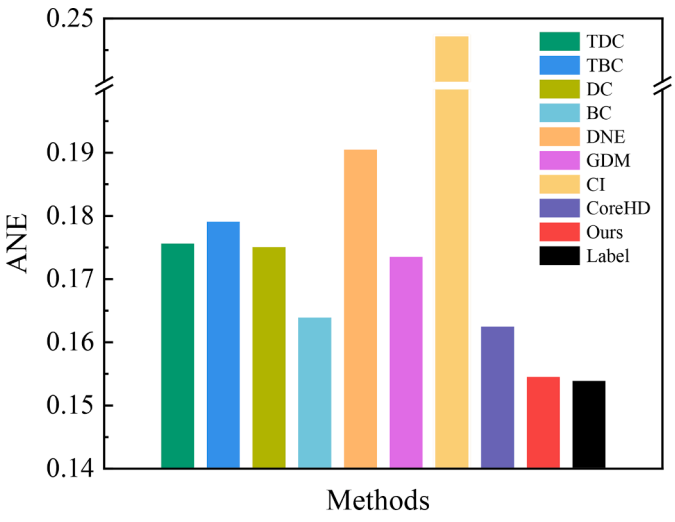

Fig. 3. Effectiveness of the methods for dismantling synthetic BA networks.

learning-based dismantling method that leverages deep learning models to identify critical nodes by learning structural patterns from network data.

First, we evaluate various methods in terms of their performance on small-scale synthetic networks. Specifically, to analyze the effectiveness of our approach across a larger number of small-scale synthetic networks, Fig. 3 shows the ANE values of various methods based on Eq. (2) for dismantling 600 synthetic BA networks in the training dataset. The size of each small synthetic network is randomly selected from the range 15 to 20. The average result of our approach is compared with the average results of the TBC, TDC, DC, BC, DNE, GDM, CI, CoreHD, and Label methods. As shown in Fig. 3, our approach outperforms the TBC, TDC, DC, BC, DNE, GDM, CI, and CoreHD methods and achieves results comparable to the Label method for 600 synthetic networks. This observation demonstrates that our approach effectively identifies the critical tuple removal sequence for efficiently dismantling temporal networks.

Moreover, our approach is trained on synthetic BA networks consisting of 15 to 20 tuples and then generalized to dismantle synthetic BA and WS networks of different scales, including 30, 50, 100, 200, 500, 1000, 2000, 5000, and 6000 tuples. For each network scale, 10 instances are randomly generated, and the average results are reported. Fig. 4 presents the ANE values on synthetic BA and WS networks calculated according to Eq. (2) to analyze the effectiveness of different methods. The results in Fig. 4 indicate that: Firstly, our approach outperforms TDC, TBC, DC, BC, GDM, CI, and CoreHD methods in generalizing to dismantle larger-scale synthetic networks. Secondly, our approach also achieves superior dismantling performance on network types different from those used during training, indicating that our approach exhibits

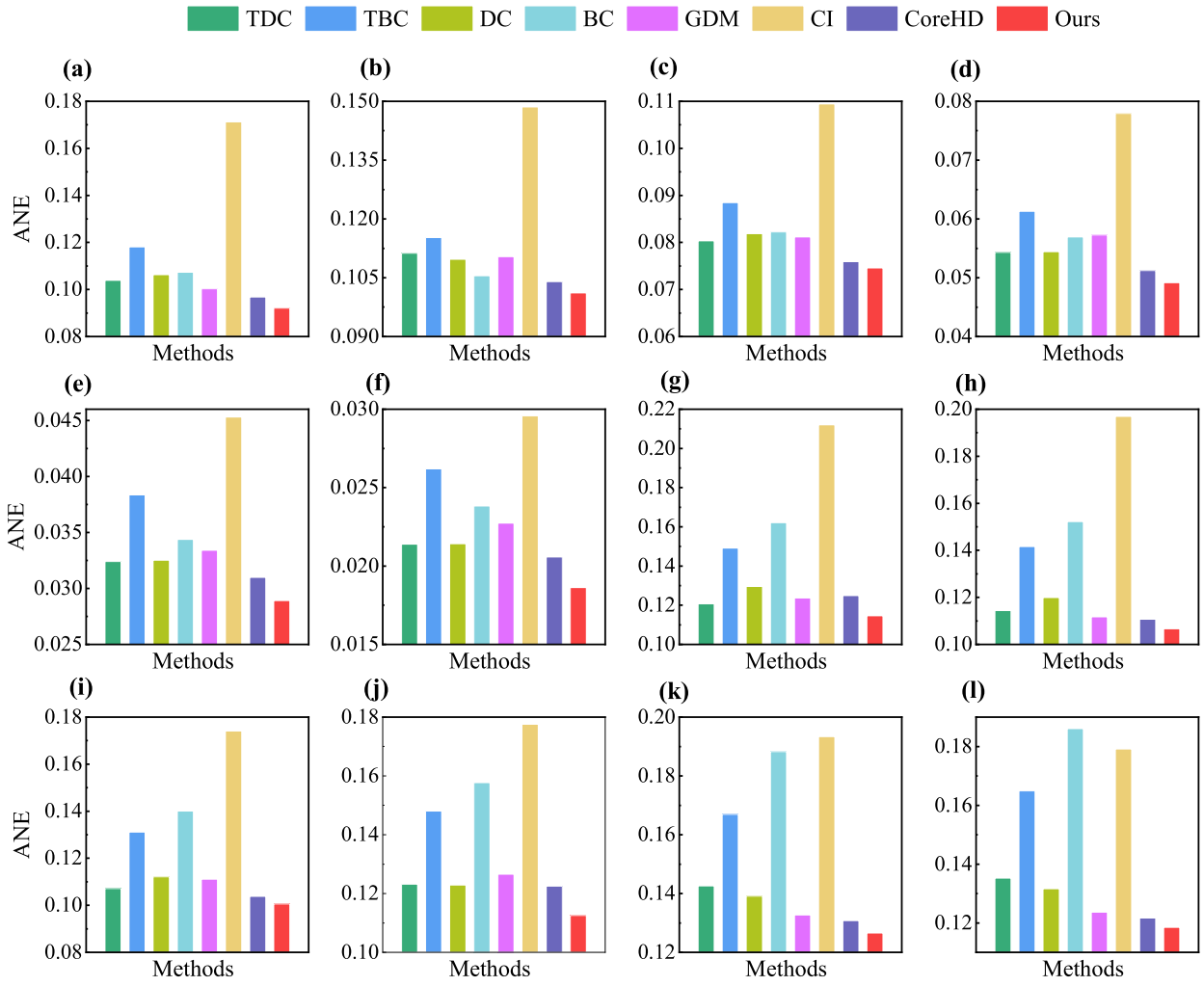

Fig. 4. Effectiveness of the methods for dismantling synthetic networks. BA networks with (a) 30 tuples, (b) 50 tuples, (c) 100 tuples, (d) 200 tuples, (e) 500 tuples, (f) 1000 tuples. WS networks with (g) 200 tuples, (h) 500 tuples, (i) 1000 tuples, (j) 2000 tuples, (k) 5000 tuples, (l) 6000 tuples.

Table 1
Statistics of the real-world networks used in the experimental analysis.

Transportation networks	Nodes	Tuples	Temporal edges	Time window
Global liner shipping network_7day	568	2416	6360	2023.6.1–2023.6.7
Global liner shipping network_14day	623	5109	19,550	2023.6.1–2023.6.14
Global liner shipping network_21day	656	7913	33,761	2023.6.1–2023.6.21
Global liner shipping network_28day	666	10,695	48,076	2023.6.1–2023.6.28
Global liner shipping network_35day	670	13,424	62,219	2023.6.1–2023.7.5
China's high-speed railway network_10hour	584	6923	31,007	0:00–10:00
China's high-speed railway network_12hour	604	13,226	96,516	0:00–12:00
China's high-speed railway network_16hour	616	25,593	345,565	0:00–16:00
China's high-speed railway network_24hour	624	44,483	1057,690	0:00–24:00
Stockholm commuter train network_12hour	44	779	2621	0:00–12:00
Stockholm commuter train network_24hour	45	2341	20,730	0:00–24:00
Indian airline network_6hour	65	1498	10,488	6:00–12:00
Indian airline network_12hour	72	3833	81,356	6:00–18:00
Beijing subway network_2hour	103	6033	17,611	6:00–8:00
Beijing subway network_6hour	103	20,549	67,188	6:00–12:00
Beijing subway network_12hour	103	41,792	138,129	6:00–18:00

strong robustness.

To more intuitively demonstrate the influence of different attack methods on network efficiency through the tuple removal, Fig. B.2 of Appendix B illustrates the performance of 12 synthetic networks under various attack strategies at tuple sizes of 30, 50, 100, 200, 500, 1000, 2000, 5000, and 6000. The fraction of removed tuples is shown on the horizontal axis, while the resulting network efficiency is shown on the vertical axis. The values in the legend correspond to the ANE values of the synthetic networks using varying attack strategies. As shown in Fig. B.2, relative to other approaches, our approach exhibits the earliest and fastest decline in network efficiency after the removal of attack tuples in most instances. This phenomenon indicates that our approach effectively identifies the removal sequence of critical tuples within the network and achieves network collapse by removing only a small number of tuples.

4.2. *Effectiveness comparison on real-world networks*

The real-world transportation schedule data used in this study are described as follows. (1) Global liner shipping schedule data: This study collects each vessel's trajectory data and shipping schedule information, including port calls and corresponding timestamps, which cover 728 ports worldwide. (2) China's high-speed railway schedule data: This study employs China's 2016 high-speed railway daily timetable as its primary data source. This data is sourced from GitHub[1] and includes 4458 trains, encompassing 672 high-speed rail stations across the country. (3) Stockholm commuter train schedule data: We use Stockholm commuter train service data from a weekday in 2012, sourced from Kaggle.[2] (4) Indian airline schedule data: We use the Indian flight schedules dataset from 2018 to 2021 as the raw data for our work. This data, originally collected from https://www.data.gov.in, is available at Kaggle.[3] The dataset includes 9 airlines and 11,976 flights. (5) Beijing subway schedule data: We use the schedule data of Beijing subway lines 1, 2, 5, and 6 from one weekend day, sourced from GitHub,[4] as the raw data for our experiments. We model the real-world transportation schedule data as a temporal transportation network based on a directed acyclic graph topology representation, enabling a complete representation of the continuous topological changes of the temporal transportation network. Table 1 shows the statistical characteristics of real-world transportation networks utilized in our experimental analysis, including the number of nodes, tuples, temporal edges, and the time window.

We examine the effectiveness of different methods on real-world transportation systems. Fig. 5 presents the ANE values for 16 real-world networks evaluated using different methods. The results in Fig. 5 indicate that our approach (Ours) achieves superior performance over TDC, TBC, DC, BC, DNE, GDM, CI, and CoreHD methods across the real-world transportation networks. In terms of the average ANE over 16 real-world networks, our approach outperforms the TDC, TBC, DC, BC, GDM, CI, and CoreHD methods by 24.77 %, 52.9 %, 19.91 %, 52.22 %, 25.88 %, 22.15 %, and 12.98 %, respectively. Then, for dismantling 16 real-world networks, compared with the TDC method, our approach achieves the highest optimality when solving the Beijing subway network_12hour, and the lowest optimality when solving the China's high-speed railway network_10hour. Compared with the TBC method, our approach reaches the highest optimality when solving the Beijing subway network_6hour, and the lowest optimality when solving the Indian airline network_12hour. Compared with the DC method, our approach obtains the highest optimality when solving the Stockholm commuter train network_24hour, and the lowest optimality when solving the Beijing subway network_2hour. Compared with the BC method, our approach attains the highest optimality when solving the Beijing subway network_6hour, and the lowest optimality when solving the Indian airline network_6hour. Compared with the GDM method, our approach achieves the highest optimality when solving the Beijing subway network_12hour, and the lowest optimality when solving the Global liner shipping network_7day. Compared with the CI method, our approach attains the highest optimality when solving the Indian airline network_12hour, and the lowest optimality when solving the Stockholm commuter train network_12hour. Compared with the CoreHD method, our approach reaches the highest optimality when solving the Stockholm commuter train network_24hour, and the lowest optimality when solving the China's high-speed railway network_12hour. Due to the high computational complexity of the DNE method, the maximum network size that can be handled in real-world applications is limited to 5109. As shown in Fig. 5, the DNE method was applied to six real-world transportation networks, where our approach consistently outperforms the DNE method. Specifically, compared with the DNE method, our approach achieves the highest optimality when solving the Stockholm commuter train network_24hour, and the lowest optimality when solving the Global liner shipping network_14day.

To more intuitively illustrate the impact of attacking tuples using different methods on dismantling temporal transportation networks, Fig. 6 presents network efficiency of the above 16 real-world networks under various attack strategies. The horizontal axis indicates the fraction of removed tuples. The vertical axis indicates the post-removal network efficiency normalized by its initial value. The legend values correspond to the ANE values using varying attack strategies. The gray dashed lines

[1] https://github.com/metromancn/Parse12306

[2] https://www.kaggle.com/datasets/abdeaitali/commuter-train-timetable

[3] https://www.kaggle.com/datasets/nikhilkhetan/indian-flight-schedules

[4] https://github.com/BoyInTheSun/beijing-subway-schedule

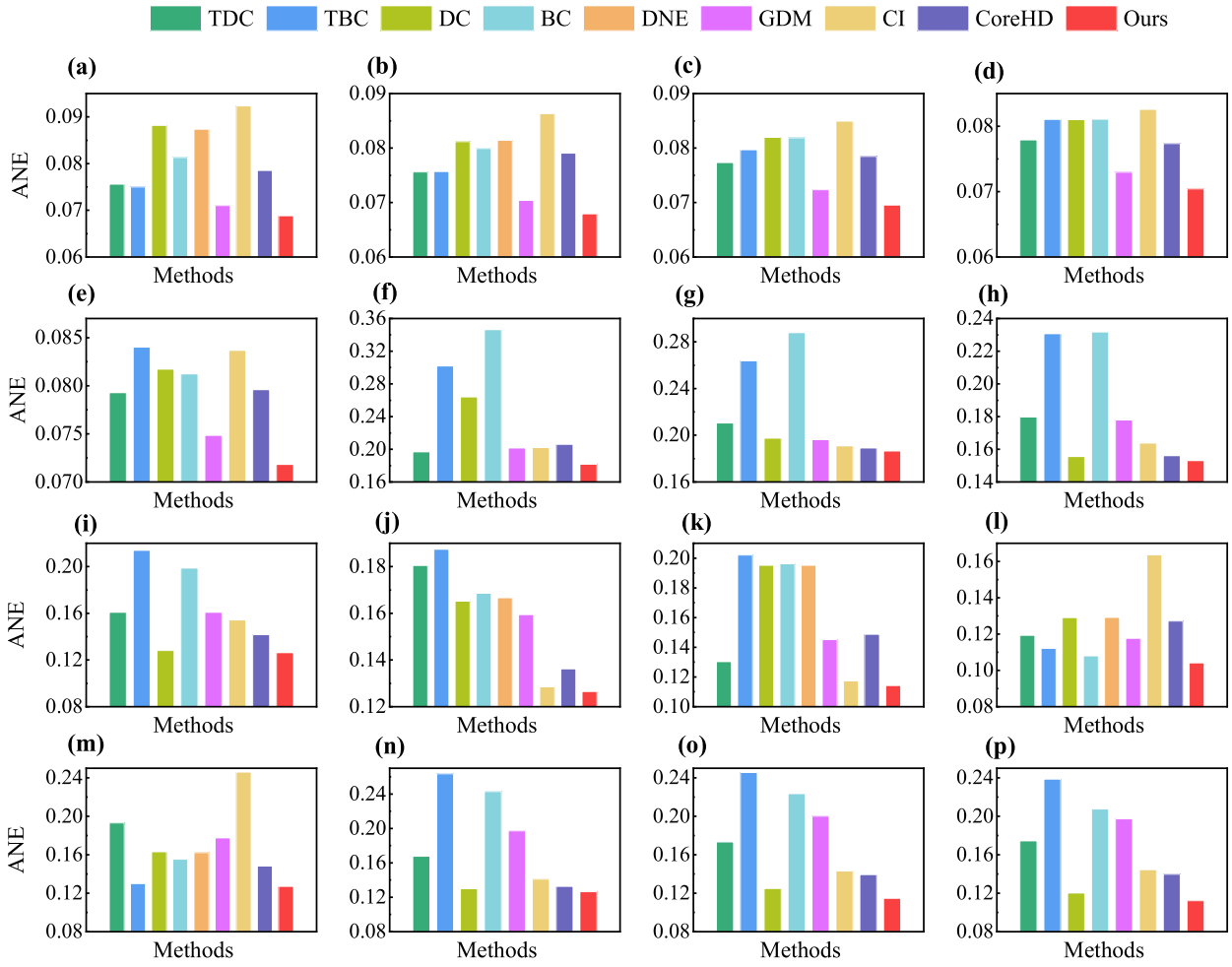

Fig. 5. Effectiveness of the methods for dismantling real-world temporal transportation networks. (a) Global liner shipping network_7day. (b) Global liner shipping network_14day. (c) Global liner shipping network_21day. (d) Global liner shipping network_28day. (e) Global liner shipping network_35day. (f) China's high-speed railway network_10hour. (g) China's high-speed railway network_12hour. (h) China's high-speed railway network_16hour. (i) China's high-speed railway network_24hour. (j) Stockholm commuter train network_12hour. (k) Stockholm commuter train network_24hour. (l) Indian airline network_6hour. (m) Indian airline network_12hour. (n) Beijing subway network_2hour. (o) Beijing subway network_6hour. (p) Beijing subway network_12hour.

in Fig. 6 indicate that the network efficiency is dismantled to 0.1, following the dismantling target of the training labels. As shown in Fig. 6, when the network efficiency is dismantled to 0.1, our approach achieves the earliest and fastest degradation in network efficiency upon the removal of targeted tuples in most instances, compared with other methods. Moreover, when the network efficiency is completely dismantled, our approach outperforms the others. These observations suggest that our approach can effectively identify the removal sequence of a small number of critical tuples and achieve efficient dismantling of the network.

Additionally, taking the Global liner shipping network_21day as an example, we further analyze the top 30 tuples identified by our approach in the network. Table 2 presents the top 30 most critical (node, time) tuples identified using our approach. From Table 2, it can be observed that June 8, 2023, is the most important day (rank 1) for six ports, while June 9, 2023, is the second most important day (rank 2) for four ports. Then, in the 21-day global liner shipping network, the port of Singapore appeared for 15 days. Moreover, all Top 5 tuples are for the port of Singapore, occurring on June 13, 2023; June 8, 2023; June 11, 2023; June 12, 2023; and June 7, 2023. This indicates that Singapore is the most important port for maintaining network efficiency. The ports of Rotterdam and Antwerp each appear for 3 days, indicating that they are relatively important ports for maintaining network efficiency.

4.3. Ablation study

We first validate our model's performance with the shortcut connections technique (SC). Both the proposed model and the model without SC are trained using supervised learning, and they are referred to as Ours and Ours(-SC), respectively. The model's performance is evaluated based on the behavior of the loss function during training, with the corresponding descent curve presented in Fig. 7. The training epochs (up to 300) are shown on the horizontal axis. The training loss value per epoch is shown on the vertical axis. As clearly shown in Fig. 7 (a), Ours exhibits superior convergence relative to Ours(-SC). The primary advantage of faster convergence is that stable performance can be reached with fewer training iterations, which in turn enhances both time and computational efficiency. Notably, the training error of Ours is significantly lower than that of Ours(-SC), indicating that the integration of the shortcut connections technique effectively mitigates the vanishing

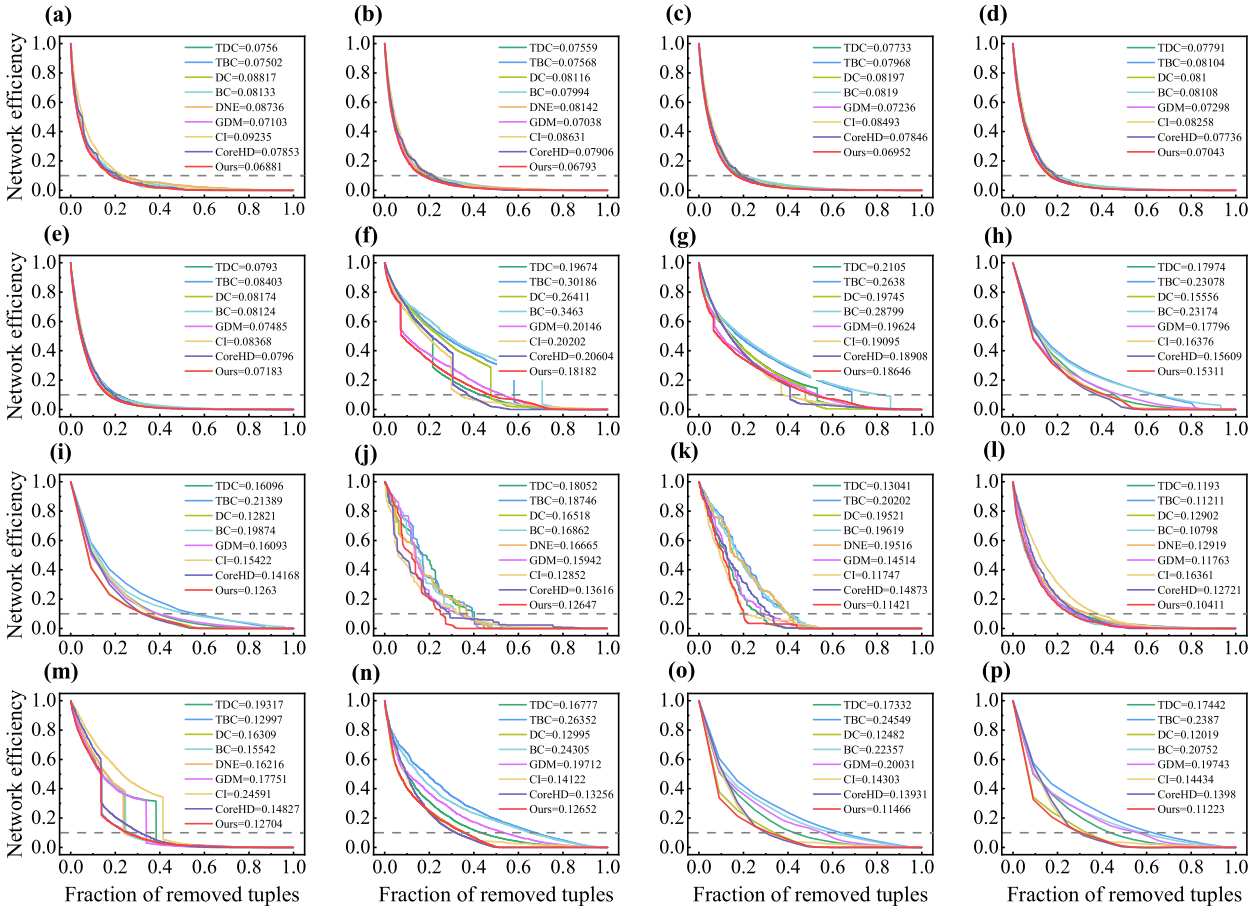

Fig. 6. Network efficiency of real-world networks under different methods. (a) Global liner shipping network_7day. (b) Global liner shipping network_14day. (c) Global liner shipping network_21day. (d) Global liner shipping network_28day. (e) Global liner shipping network_35day. (f) China's high-speed railway network_10hour. (g) China's high-speed railway network_12hour. (h) China's high-speed railway network_16hour. (i) China's high-speed railway network_24hour. (j) Stockholm commuter train network_12hour. (k) Stockholm commuter train network_24hour. (l) Indian airline network_6hour. (m) Indian airline network_12hour. (n) Beijing subway network_2hour. (o) Beijing subway network_6hour. (p) Beijing subway network_12hour. Note: The values in (a)–(p) legends represent the ANE values of networks using different methods. The gray dashed lines in (a)–(p) indicate that the network efficiency is dismantled to 0.1, following the dismantling target of the training labels.

gradient issue.

Then, we analyze the impact of input features on our model during the training process. We input both TDC and TBC features into our model, referred to as Ours(TDC, TBC). We also consider only TDC or only TBC features, referred to as Ours(TDC) and Ours(TBC), respectively. Ours(TDC, TBC), Ours(TDC), and Ours(TBC) are all trained using supervised learning. As clearly shown in Fig. 7(b), Ours(TDC, TBC) exhibits faster convergence compared to Ours(TDC) and Ours(TBC), with Ours(TBC) converging faster than Ours(TDC). Moreover, the training error of Ours(TDC, TBC) is significantly lower than that of Ours(TDC) and Ours(TBC), and the training error of Ours(TDC) is lower than that of Ours(TBC). This indicates that the combination of TDC and TBC features demonstrates a powerful synergistic effect, significantly enhancing the model's training performance. Meanwhile, the TDC feature alone shows a more significant advantage in enhancing training effectiveness compared to the TBC feature.

Finally, to further analyze the effectiveness and generalization of pre-trained model on the test dataset, we use synthetic networks of different types and scales as the test dataset. Fig. 8 presents the ANE values calculated for 12 synthetic BA and WS networks according to Eq. (2) using TDC, TBC, Ours(TDC), Ours(TBC), and Ours(TDC, TBC). As observed in Fig. 8, Ours(TDC, TBC) outperforms TDC, TBC, Ours(TDC), and Ours(TBC) for dismantling 12 synthetic networks, indicating that our model can incorporate different temporal features to rapidly dismantle networks. Furthermore, for the dismantling results of 12 instances, Ours(TDC) outperforms TDC, and Ours(TBC) outperforms TBC, indicating that the proposed model can capture latent higher-order temporal features even on previously unseen networks.

4.4. *Early warning signals of network dismantling risk*

For critical real-world transportation systems, accurately assessing the importance of each node in advance at different time can provide early warnings of potential network performance dismantling risks under arbitrary node failure scenarios. By comparing node failure scenarios under the lower bound of network performance collapse, the early-warning signals [31,61] are developed to assess the dismantling risk of network performance in node failure scenarios.

Our approach calculates the tuples' dismantling scores, enabling the prediction of the potential impact of future attacks on network efficiency. Inspired by the work of [31], we use an Early Warning Signal (EWS) indicator to quantify network dismantling risk. This early

Table 2
Top 30 most critical tuples in the global liner shipping network.

Rank	(node, time) tuple	Country	Rank	(node, time) tuple	Country
1	(Singapore, 2023–06–13)	Singapore	16	(Singapore, 2023–06–17)	Singapore
2	(Singapore, 2023–06–08)	Singapore	17	(Singapore, 2023–06–15)	Singapore
3	(Singapore, 2023–06–11)	Singapore	18	(Singapore, 2023–06–04)	Singapore
4	(Singapore, 2023–06–12)	Singapore	19	(Singapore, 2023–06–06)	Singapore
5	(Singapore, 2023–06–07)	Singapore	20	(Antwerp, 2023–06–11)	Belgium
6	(Chiwan, 2023–06–10)	China	21	(Singapore, 2023–06–20)	Singapore
7	(Singapore, 2023–06–16)	Singapore	22	(Rotterdam, 2023–06–17)	Netherlands
8	(Rotterdam, 2023–06–15)	Netherlands	23	(Nansha, 2023–06–09)	China
9	(Singapore, 2023–06–14)	Singapore	24	(Busan, 2023–06–14)	South Korea
10	(Singapore, 2023–06–09)	Singapore	25	(Ningbo zhoushan, 2023–06–08)	China
11	(Perama, 2023–06–08)	Greece	26	(Waigaoqiao, 2023–06–08)	China
12	(Singapore, 2023–06–10)	Singapore	27	(Antwerp, 2023–06–09)	Belgium
13	(Antwerp, 2023–06–08)	Belgium	28	(Ningbo zhoushan, 2023–06–12)	China
14	(Singapore, 2023–06–05)	Singapore	29	(Rotterdam, 2023–06–08)	Netherlands
15	(Busan, 2023–06–18)	South Korea	30	(Hong kong, 2023–06–09)	China

warning signal assists stakeholders in formulating policies and decisions, particularly in transportation systems facing potential failures or targeted attacks. Given a tuple disruption scenario, we can get a corresponding EWS value defined by Eq. (8). EWS values approaching 1 are correlated with elevated network dismantling risks.

$$\mathrm{EWS} = \begin{cases} \dfrac{\sum_{i\in S} dis\_score_i}{\sum_{i\in S*} dis\_score_i} \ if \ \sum_{i\in S} dis\_score_i \le \sum_{i\in S*} dis\_score_i \\ 1 \ if \ \sum_{i\in S} dis\_score_i > \sum_{i\in S*} dis\_score_i \end{cases} \tag{8}$$

where $S*$ represents the optimal (or predicted) attacked set identified by our model, which aims to maximize the degradation of temporal network efficiency; S is an attacked tuple set, i.e., a set of (node, time) tuples selected for removal; dis_score denotes tuples dismantling scores predicted by our model; $\sum_{i\in S} dis\_score_i$ indicates the level of degradation suffered by the network under arbitrary tuple attack scenarios; $\sum_{i\in S*} dis\_score_i$ indicates the level of degradation the network can withstand before its performance degrades to a given lower bound δ (such as the extent to which network efficiency is dismantled, i.e., the δ is set to 0, 0.1, 0.2, and 0.3 in our experiments) under targeted removal of crucial tuples.

Taking the Global liner shipping network_21day as an example, Fig. 9 illustrates the EWS indicator calculated according to Eq. (8) under two targeted attack strategies (e.g., TDC, TBC) and the δ is set to 0, 0.1, 0.2, and 0.3. The fraction of removed tuples is shown on the horizontal axis. The left vertical axis indicates network efficiency, while the right vertical axis tracks the value of the EWS as the fraction of removed tuples increases. As can be seen from Fig. 9, although network efficiency exhibits a declining trend when the system is under attack, it fails to provide a clear warning signal timely until the system is severely damaged and eventually collapses. The EWS value increases more rapidly when critical tuples are attacked. The EWS value reaches the warning level before the system degrades to a given lower bound δ, which can be reflected by the first response time. The initial response time is characterized by the time interval between the network efficiency being degraded to a given lower bound δ and the early warning signal reaching 50% (i.e., EWS = 0.5). Specifically, when δ is set to 0.3, TDC and TBC remove the top 8.43% and 8.16% of tuples respectively; under TDC and TBC attacks, the EWS reaches 0.5 when the top 2.98% and 3.21% of tuples are removed respectively. When δ is set to 0.2, TDC and TBC remove the top 11.42% and 12.14% of tuples respectively; under TDC and TBC attacks, the EWS reaches 0.5 when the top 4.02% and 4.28% of tuples are removed respectively. When δ is set to 0.1, TDC and TBC remove the top 17.97% and 20.35% of tuples respectively; under TDC and TBC attacks, the EWS reaches 0.5 when the top 5.86% and 6.18% of tuples are removed respectively. When δ is set to 0, TDC and TBC remove the top 86.3% and 97.51% of tuples respectively; under TDC and TBC attacks, the EWS reaches 0.5 when the top 32.77% and 34.13% of tuples are removed respectively. Additionally, the first-order derivative of the EWS can track the importance of the attacked tuple, thereby revealing how the attack intensity evolves over time. In conclusion, our EWS indicator can provide timely decision-making information for stakeholders so as to reduce economic losses.

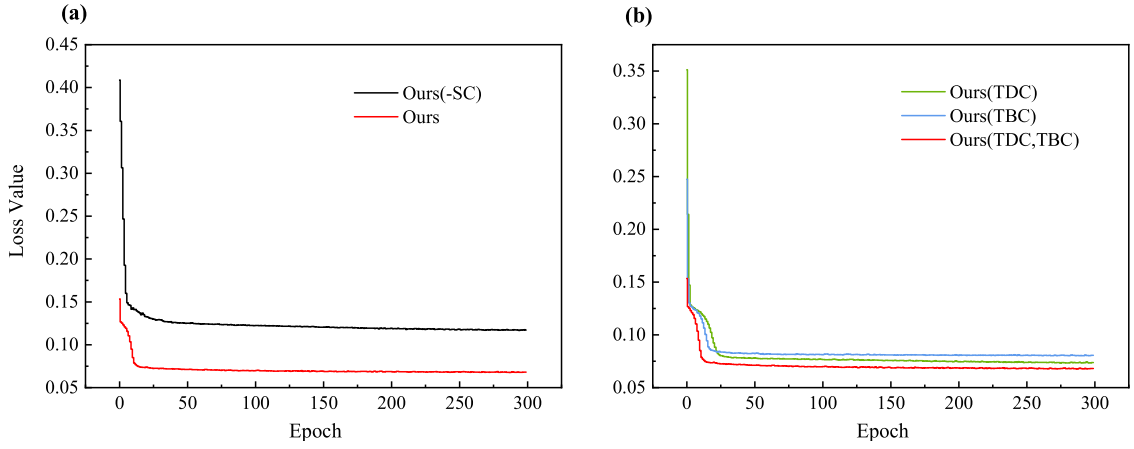

Fig. 7. Visualizations of the training loss decline curves. (a) Validation of model performance with the SC. (b) Impact assessment of input features on model performance.

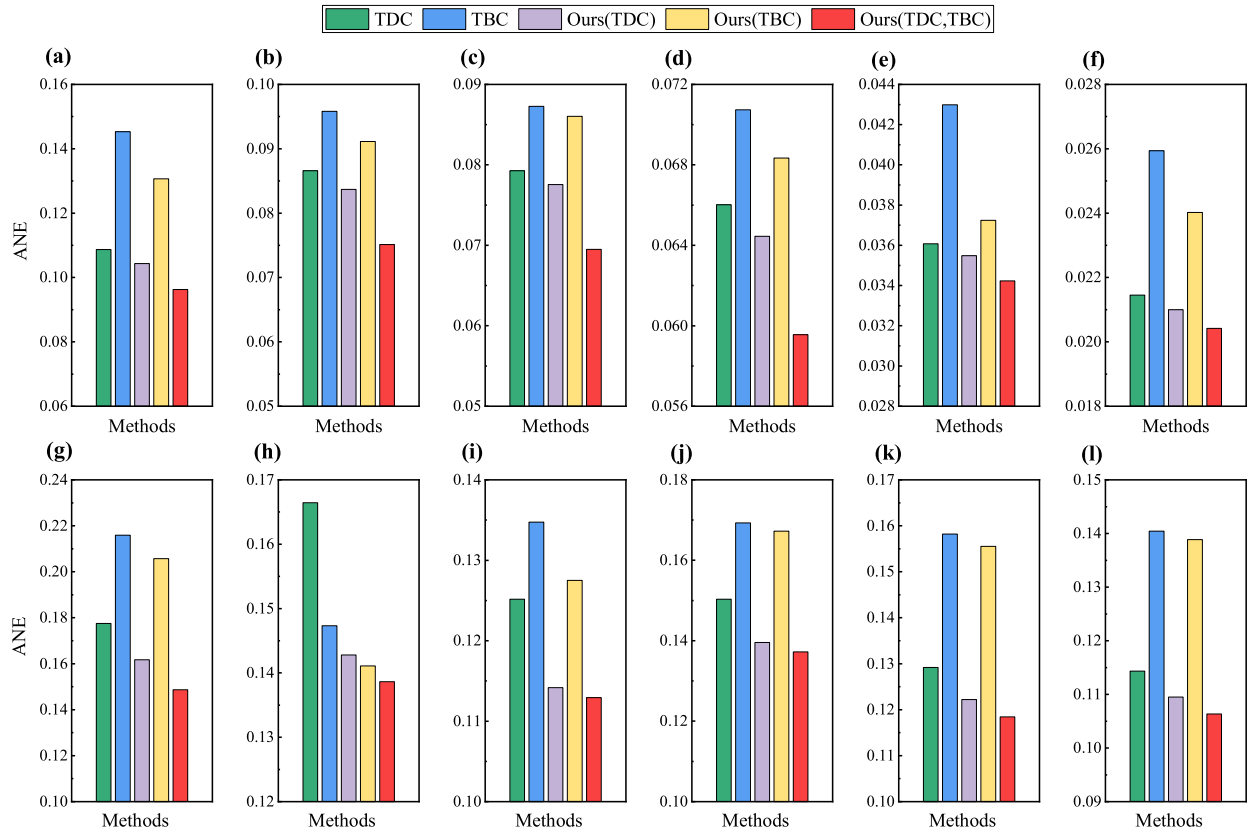

Fig. 8. Performance comparison of TDC, TBC, Ours(TDC), Ours(TBC), and Ours(TDC, TBC) for different instances. BA networks with (a) 30 tuples, (b) 50 tuples, (c) 100 tuples, (d) 200 tuples, (e) 500 tuples, (f) 1000 tuples. WS networks with (g) 30 tuples, (h) 50 tuples, (i) 100 tuples, (j) 200 tuples, (k) 500 tuples, (l) 1000 tuples.

5. Conclusions

It is a crucial issue to accurately assess the significance of individual nodes at different time for maintaining the efficiency of real-world temporal transportation networks. This paper models this issue as a temporal ND problem, which aims to find the minimal set of nodes with time information whose removal can rapidly degrade network efficiency. We formulate a mathematical model for such temporal ND problem, and then develop a deep-learning-based network dismantling method for its solution. Concretely, in formulating the temporal ND problem model, we adopt a directed acyclic graph representation of temporal network topology, which fully captures the continuous topological changes in temporal transportation networks and provides the foundation for quantifying network efficiency. Our proposed deep-learning-based dismantling method employs the multi-head GATs to capture the latent temporal features of a network and incorporates the SC technique to mitigate the vanishing gradient issue, thus strengthening the model's representational power for learning complex patterns. The proposed model generates the higher-order features of nodes through aggregation of different temporal features of nodes and their neighbors, which are then used to obtain the dismantling score for any node at any time. Extensive experiments on various synthetic networks and real-world transportation systems demonstrate the superiority and strong robustness of our approach over temporal centrality metrics of degree and betweenness. Our approach enables accurate assessment of the significance of individual nodes at different time for maintaining network efficiency. Besides, our approach can identify critical nodes with time information in temporal transportation networks, whose disruption would most severely degrade network efficiency. It provides actionable insights for vulnerability assessment, infrastructure protection, and emergency response planning. In practice, our approach can help transportation operators and policymakers prioritize monitoring, maintenance, and intervention strategies based on node importance when resources are limited, thereby supporting data-driven risk mitigation and resilience enhancement in operational transportation systems.

The dismantling scores of individual nodes at different time calculated by our proposed method are used to predict the influence of future node failures or attacks on transportation network efficiency. As one important example of such application, we derive an early warning signal indicator from the dismantling scores to quantify network dismantling risk. It offers a practical tool for identifying key nodes within temporal transportation networks, thereby assisting stakeholders in enhancing the robustness of transportation services, especially in scenarios of unforeseen disasters or accidents.

While the proposed approach utilizes small synthetic networks for training and demonstrates strong generalization capabilities on various large-scale synthetic networks and real-world transportation networks, one limitation is its reliance on supervised learning. Due to the NP-hard nature of temporal network dismantling, obtaining optimal solutions for supervised training is highly challenging and time-consuming, even for small-scale networks. Considering the advantages of reinforcement learning, which eliminates the need for manually labeled data and allows the model to learn strategies autonomously through environmental interactions, it is promising for future studies to employ reinforcement learning methods for addressing the temporal network dismantling problem. Additionally, some previous studies [62,63] indicate that community structures and hierarchical features play important roles in the resilience of real-world transportation networks. Community center nodes, such as transportation hubs, often connect multiple high-density

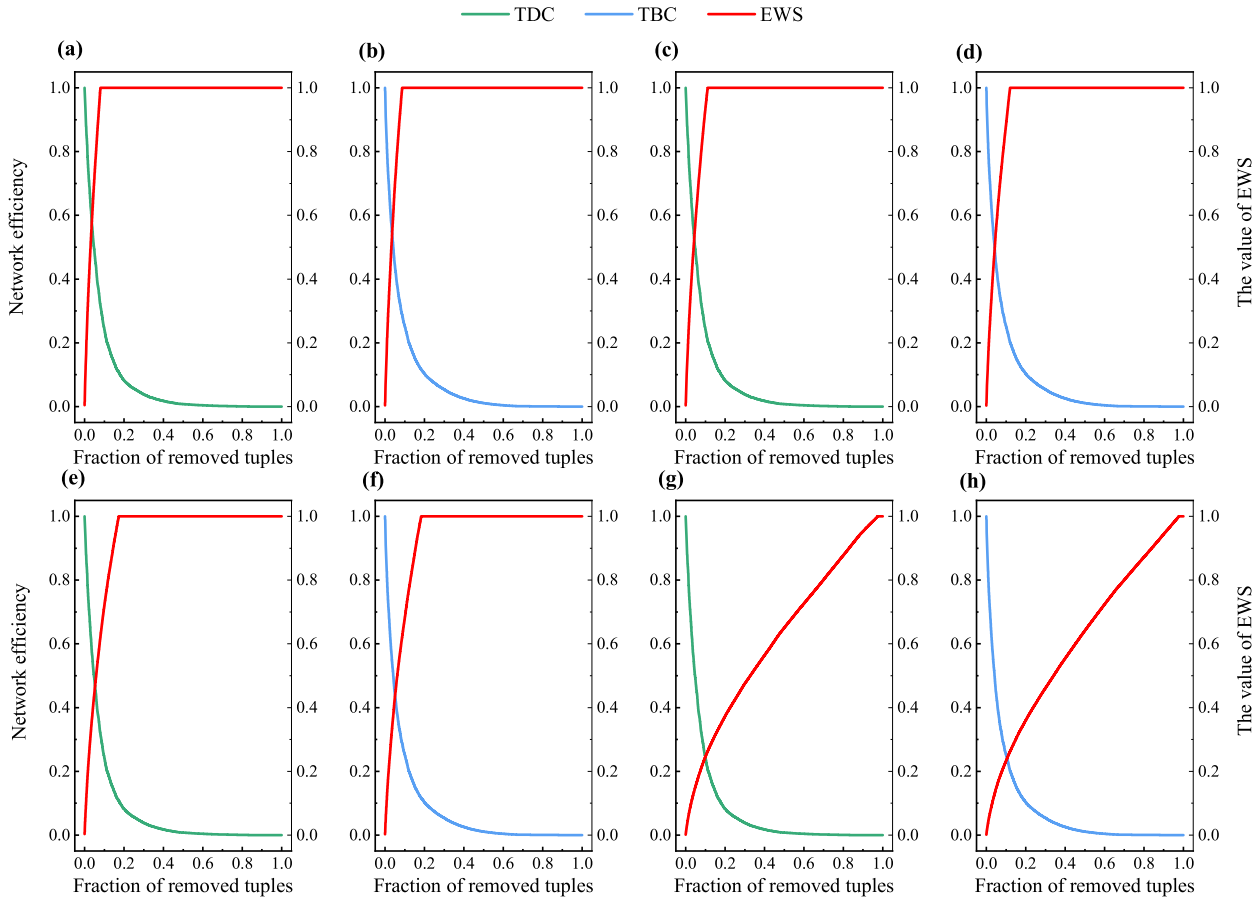

Fig. 9. Early warning signals of network dismantling under different targeted attacks. (a) TDC attack strategy and $\delta = 0.3$. (b) TBC attack strategy and $\delta = 0.3$. (c) TDC attack strategy and $\delta = 0.2$. (d) TBC attack strategy and $\delta = 0.2$. (e) TDC attack strategy and $\delta = 0.1$. (f) TBC attack strategy and $\delta = 0.1$. (g) TDC attack strategy and $\delta = 0$. (h) TBC attack strategy and $\delta = 0$.

sub-networks, and their failure significantly weakens network connectivity and network efficiency. Meanwhile, the hierarchical features of transportation networks mean that disruptions to higher-level facilities (e.g., highways, subways, etc.) tend to have more severe impacts on network efficiency. Although our approach does not explicitly incorporate community structure as a prior feature, the temporal feature aggregation mechanism based on multi-head graph attention networks can implicitly capture the mesoscopic cluster structure within the network. Therefore, nodes that play the role of community centers or inter-community bridges within a given time window are more likely to be assigned higher dismantling scores, reflecting their key role in maintaining network efficiency. In future work, explicitly incorporating community structures and hierarchical features into our approach represents an important direction for further improving the performance of temporal transportation network dismantling.

Data availability statement

The code and data that support the findings of this study will be made available on request when the paper is accepted for publication.

CRediT authorship contribution statement

Li Hong: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Yifan Zhu:** Writing – review & editing, Software, Methodology, Investigation, Data curation. **Kaishuo Liu:** Validation, Software, Methodology, Investigation, Data curation. **Yu Liu:** Writing – review & editing, Resources, Funding acquisition. **Changjun Fan:** Writing – review & editing, Methodology, Funding acquisition. **Mengqiao Xu:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

M.X. acknowledges the support from the National Natural Science Foundation of China (Grant: 72571047; 72101046). Y.L. acknowledges the support from the National Natural Science Foundation of China (Grant: 61672128) and from the Dalian Key Field Innovation Team Support Plan (Grant: 2020RT07). C.F. acknowledges the support from the National Natural Science Foundation of China (NSFC, 72571284, 62206303), Hunan Provincial Fund for Distinguished Young Scholars (2025JJ20073), Science and Technology Innovation Program of Hunan Province (2023RC3009).

Appendix A

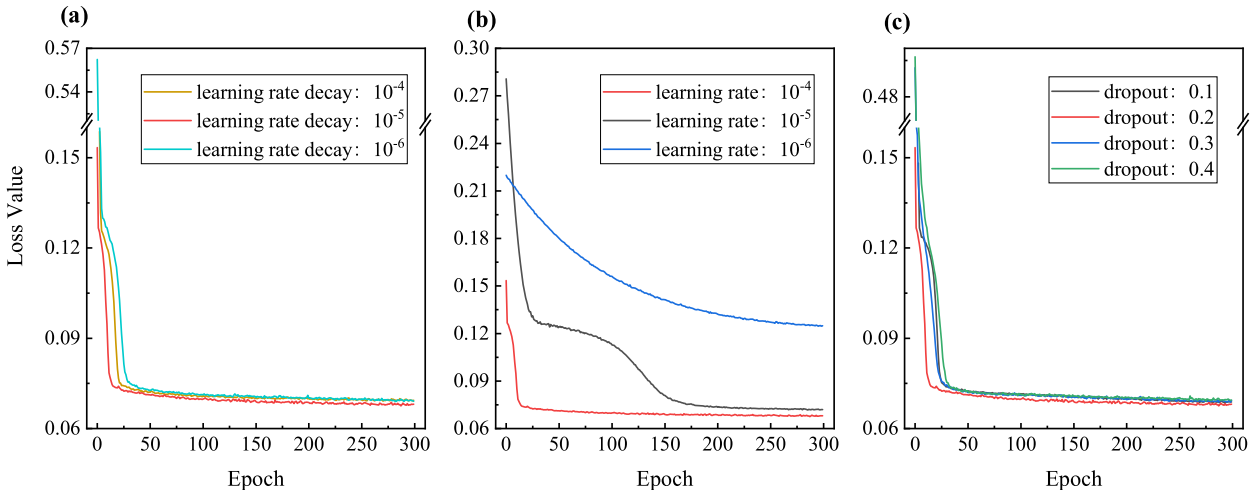

Fig. A.1. Impact of hyperparameter settings on model performance, including (a) learning rate decay, (b) the learning rate of Adam optimizer, and (c) probability of dropout.

Appendix B

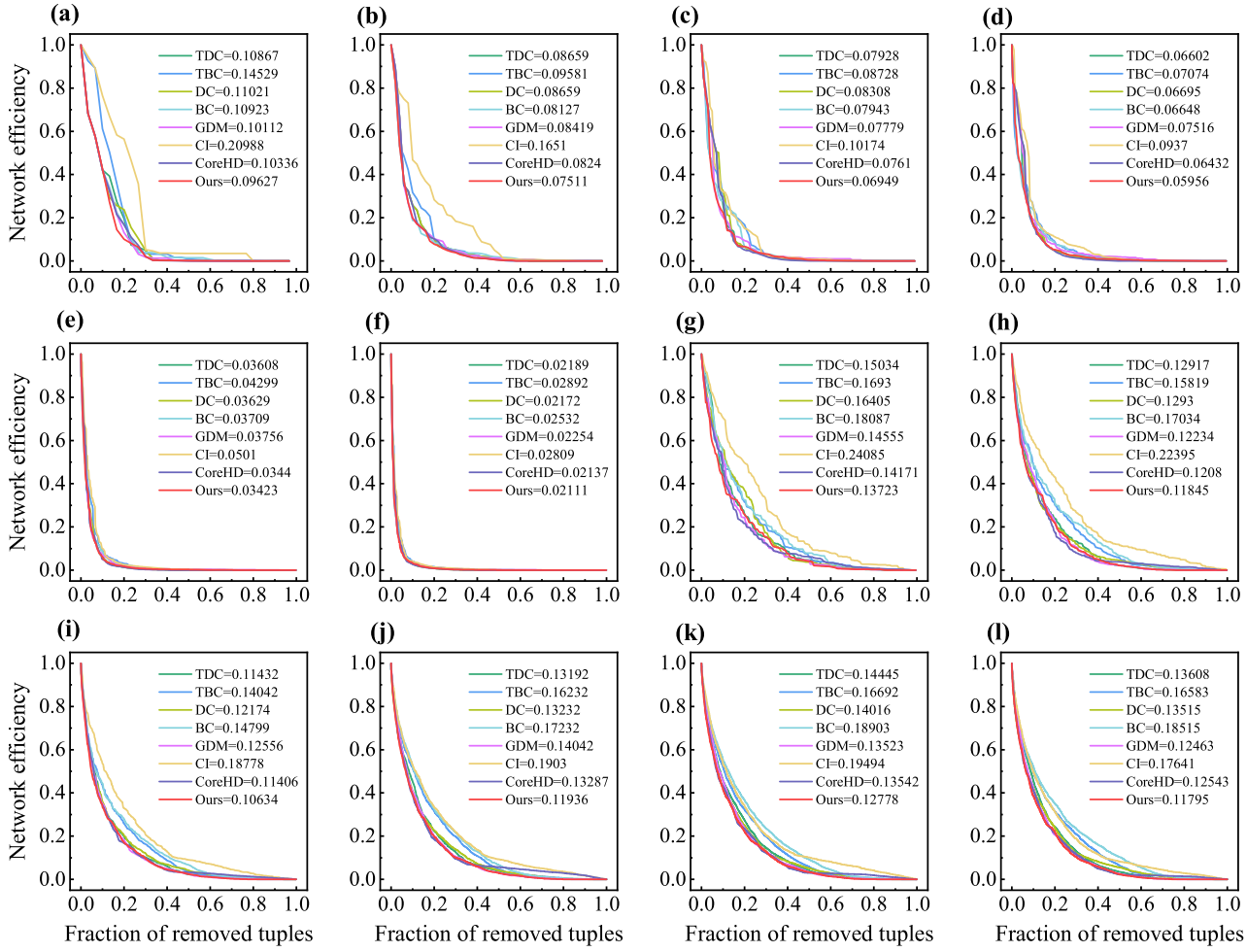

Fig. B.2. Network efficiency of synthetic networks under different methods. BA networks with (a) 30 tuples, (b) 50 tuples, (c) 100 tuples, (d) 200 tuples, (e) 500 tuples, (f) 1000 tuples. WS networks with (g) 200 tuples, (h) 500 tuples, (i) 1000 tuples, (j) 2000 tuples, (k) 5000 tuples, (l) 6000 tuples. Note: The values in the (a)–(l) legends represent the ANE values of networks using different methods.

References

[1] Xu M, Pan Q, Muscoloni A, Xia H, Cannistraci CV. Modular gateway-ness connectivity and structural core organization in maritime network science. Nat Commun 2020;11(1):2849. https://doi.org/10.1038/s41467-020-16619-5.

[2] Zhou H, Zhou L, Guo B, Bai Z, Wang Z. Analyzing the railway operation network by evaluating the importance of time-space nodes. IEEE Trans Netw Sci Eng 2022;9 (6):4209–19. https://doi.org/10.1109/TNSE.2022.3196397.

[3] Kang L, Buhigiro N, Sun H, Wu J. A critical review of subway train timetabling and rescheduling problems. IEEE Trans Intell Transp Syst 2024;25(10):12930–42. https://doi.org/10.1109/TITS.2024.3386728.

[4] Abdelghany A, Abdelghany K, Azadian F. Airline flight schedule planning under competition. Comput Oper Res 2017;87:20–39. https://doi.org/10.1016/j.cor.2017.05.013.

[5] Holme P, Saramäki J. Temporal networks. Phys Rep 2012;519(3):97–125. https://doi.org/10.1016/j.physrep.2012.03.001.

[6] Trajanovski S, Scellato S, Leontiadis I. Error and attack vulnerability of temporal networks. Phys Rev - Stat Nonlinear Soft Matter Phys 2012;85(6):1–10. https://doi.org/10.1103/PhysRevE.85.066105.

[7] Zou L, Wang H. Vulnerability of information transport on temporal networks to link removal. IEEE Trans Netw Sci Eng 2025;12(4):2932–41. https://doi.org/10.1109/TNSE.2025.3555409.

[8] Sano HH, Berton L. Topology and robustness analysis of temporal air transport network. In: Proceedings of the 35th Annual ACM Symposium on Applied Computing; 2020. p. 1881–4. https://doi.org/10.1145/3341105.3374062.

[9] Williams MJ, Musolesi M. Spatio-temporal networks: reachability, centrality and robustness. R Soc open Sci 2016;3(6):160196.

[10] Li Y, Zhao Y, Xu T, Wu S. Node importance research of temporal CPPS networks for information fusion. IEEE Trans Reliab 2024;73(2):1291–301. https://doi.org/10.1109/TR.2023.3329124.

[11] Shi J, Tang Z, Zhan X, Liu C. Temporal modeling and resilience analysis of supply chain networks under cascading failures. Reliab Eng Syst Saf 2026;266(PB): 111763. https://doi.org/10.1016/j.ress.2025.111763.

[12] Feng D, Bin H, Lai J. Airports network vulnerability analysis based on temporal network. In: 2022 IEEE/AIAA 41st digital avionics systems conference (DASC). IEEE; 2022. p. 1–7. https://doi.org/10.1109/DASC55683.2022.9925725.

[13] Kim H, Anderson R. Temporal node centrality in complex networks. Phys Rev - Stat Nonlinear Soft Matter Phys 2012;85(2):026107. https://doi.org/10.1103/PhysRevE.85.026107.

[14] Tsalouchidou I, Baeza-Yates R, Bonchi F, Liao K, Sellis T. Temporal betweenness centrality in dynamic graphs. Int J Data Sci Anal 2020;9(3):257–72. https://doi.org/10.1007/s41060-019-00189-x.

[15] Zhang T, et al. Efficient exact and approximate betweenness centrality computation for temporal graphs. In: Proceedings of the ACM web conference 2024; 2024. p. 2395–406. https://doi.org/10.1145/3589334.3645438.

[16] Nicosia V, Tang J, Mascolo C, Musolesi M, Russo G, Latora V. Graph metrics for temporal networks. Temporal networks. Berlin, Heidelberg: Springer Berlin Heidelberg; 2013. p. 15–40. https://doi.org/10.1007/978-3-642-36461-7_2.

[17] Lentz HHK, Selhorst T, Sokolov IM. Unfolding accessibility provides a macroscopic approach to temporal networks. Phys Rev Lett 2013;110(11):118701. https://doi.org/10.1103/PhysRevLett.110.118701.

[18] Ye Z, Zhan X, Zhou Y, Liu C, Zhang ZK. Identifying vital nodes on temporal networks: an edge-based K-shell decomposition. In: 2017 36th Chinese control conference (CCC); 2017. p. 1402–7. https://doi.org/10.23919/ChiCC.2017.8027547.

[19] Yu EY, Fu Y, Chen X, Xie M, Chen DB. Identifying critical nodes in temporal networks by network embedding. Sci Rep 2020;10(1):12494. https://doi.org/10.1038/s41598-020-69379-z.

[20] Pitoski D, Babić K, Meštrović A. A new measure of node centrality on schedule-based space-time networks for the designation of spread potential. Sci Rep 2023;13 (1):22561. https://doi.org/10.1038/s41598-023-49723-9.

[21] Casteigts A, Flocchini P, Quattrociocchi W, Santoro N. Time-varying graphs and dynamic networks. Int J Parallel Emergent Distrib Syst 2012;27(5):387–408. https://doi.org/10.1080/17445760.2012.668546.

[22] Wu H, Huang Y, Cheng J, Li J, Ke Y. Reachability and time-based path queries in temporal graphs. In: 2016 IEEE 32nd international conference on data engineering (ICDE); 2016. p. 145–56. https://doi.org/10.1109/ICDE.2016.7498236.

[23] Wu H, Cheng J, Huang S, Ke Y, Lu Y, Xu Y. Path problems in temporal graphs. Proc VLDB Endow 2014;7(9):721–32. https://doi.org/10.14778/2732939.2732945.

[24] Gao Y, Zhang T, Qiu L, Linghu Q, Chen G. Time-respecting flow graph pattern matching on temporal graphs. IEEE Trans Knowl Data Eng 2021;33(10):3453–67. https://doi.org/10.1109/TKDE.2020.2968901.

[25] Wandelt S, Lin W, Sun X, Zanin M. From random failures to targeted attacks in network dismantling. Reliab Eng Syst Saf 2022;218(PA):108146. https://doi.org/10.1016/j.ress.2021.108146.

[26] Wandelt S, Shi X, Sun X. Estimation and improvement of transportation network robustness by exploiting communities. Reliab Eng Syst Saf 2021;206(November 2020):107307. https://doi.org/10.1016/j.ress.2020.107307.

[27] Deng Y, Wang Z, Xiao Y, Shen X, Kurths J, Wu J. Spatial network disintegration based on spatial coverage. Reliab Eng Syst Saf 2025;253(January 2025):110525. https://doi.org/10.1016/j.ress.2024.110525.

[28] Wang Z, Su Z, Deng Y, Kurths J, Wu J. Spatial network disintegration based on kernel density estimation. Reliab Eng Syst Saf 2024;245(May 2024):110005. https://doi.org/10.1016/j.ress.2024.110005.

[29] Braunstein A, Dall'Asta L, Semerjian G, Zdeborová L. Network dismantling. Proc Natl Acad Sci U S A Nov. 2016;113(44):12368–73. https://doi.org/10.1073/pnas.1605083113.

[30] Fan C, Zeng L, Sun Y, Liu YY. Finding key players in complex networks through deep reinforcement learning. Nat Mach Intell Jun. 2020;2(6):317–24. https://doi.org/10.1038/s42256-020-0177-2.

[31] Grassia M, De Domenico M, Mangioni G. Machine learning dismantling and early-warning signals of disintegration in complex systems. Nat Commun Dec. 2021;12 (1):5190. https://doi.org/10.1038/s41467-021-25485-8.

[32] Zhang J, Wang B. Dismantling complex networks by a neural model trained from tiny networks. In: International conference on information and knowledge management, proceedings. Association for Computing Machinery; Oct. 2022. p. 2559–68. https://doi.org/10.1145/3511808.3557290.

[33] Tan D, Zhang M, Shen X, Wang Z, Deng Y, Wu J. Identifying key regions in spatial networks through graph neural networks. Reliab Eng Syst Saf 2026;266(PB): 111743. https://doi.org/10.1016/j.ress.2025.111743.

[34] Zhang J, Wang B. Encoding node diffusion competence and role significance for network dismantling. In: ACM web conference 2023 - proceedings of the world wide web conference. 2023; 2023. p. 111–21. https://doi.org/10.1145/3543507.3583233. *W W W*.

[35] Hong L, Xu M, Liu Y, Zhang X, Fan C. A deep learning dismantling approach for understanding the structural vulnerability of complex networks. Chaos Solit Fractals 2025;194(December 2024):116148. https://doi.org/10.1016/j.chaos.2025.116148.

[36] Zhang W, Jiang Z, Yao Q. DND: deep learning-based directed network disintegrator. IEEE J Emerg Sel Top Circuits Syst 2023;13(3):841–50. https://doi.org/10.1109/JETCAS.2023.3290319.

[37] Jiang W, et al. Scalable rapid framework for evaluating network worst robustness with machine learning. Reliab Eng Syst Saf 2024;252(March):110422. https://doi.org/10.1016/j.ress.2024.110422.

[38] Gu W, Yang C, Li L, Hou J, Radicchi F. Deep-learning-aided dismantling of interdependent networks. Nat Mach Intell 2025;7(8):1266–77. https://doi.org/10.1038/s42256-025-01070-2.

[39] Kossinets G, Kleinberg J, Watts D. The structure of information pathways in a social communication network. In: Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining; 2008. p. 435–43.

[40] Chuanhua Z, Lichun C, Jiayi Z. Node-level resilience analysis for temporal networks based on K-shell gravity. In: 2022 IEEE international conference on unmanned systems (ICUS). IEEE; 2022. p. 1391–6. https://doi.org/10.1109/ICUS55513.2022.9986778.

[41] Enyu Y, Yan F, Junlin Z, Hongliang S, Duanbing C. Predicting critical nodes in temporal networks by dynamic graph convolutional networks. Appl Sci 2023;13 (12):7272.

[42] Bi J, Jin J, Qu C, Zhan X, Wang G, Yan G. Temporal gravity model for important node identification in temporal networks. Chaos Solit Fractals 2021;147:110934. https://doi.org/10.1016/j.chaos.2021.110934.

[43] Jarumaneeroj P, Lawton JB, Svindland M. An evolution of the global container shipping network : port connectivity and trading community structure. Marit Econ Logist 2024;26(2):283–306. https://doi.org/10.1057/s41278-023-00273-x.

[44] Kosowska-stamirowska Z. Network effects govern the evolution of maritime trade. Proc Natl Acad Sci 2020;117(23):12719–28. https://doi.org/10.1073/pnas.1906670117.

[45] Michail O. An introduction to temporal graphs: an algorithmic perspective. Internet Math 2016;12(4):239–80. https://doi.org/10.1080/15427951.2016.1177801.

[46] Tao L, et al. A sequential-path tree-based centrality for identifying influential spreaders in temporal networks. Chaos Solit Fractals 2022;165(P1):112766. https://doi.org/10.1016/j.chaos.2022.112766.

[47] Grassia M, Mangioni G. CoreGDM: geometric deep learning network decycling and dismantling. International workshop on complex networks. Cham: Springer Nature Switzerland, Springer Science and Business Media B.V; 2023. p. 86–94. https://doi.org/10.1007/978-3-031-28276-8_8.

[48] Zhong X, Liu R. Identifying critical nodes in interdependent networks by GA-XGBoost. Reliab Eng Syst Saf 2024;251(January):110384. https://doi.org/10.1016/j.ress.2024.110384.

[49] Schneider CM, Moreira AA, Andrade JS, Havlin S, Herrmann HJ. Mitigation of malicious attacks on networks. Proc Natl Acad Sci U S A 2011;108(10):3838–41. https://doi.org/10.1073/pnas.1009440108.

[50] Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y. Graph attention networks. Stat Oct. 2017;1050(20):10–48550 [Online]. Available, http://arxiv.org/abs/1710.10903.

[51] Kafadar K, Koehler JR, Venables WN, Ripley BD. Modern applied statistics with S-plus. Am Stat 1999;53(1):86. https://doi.org/10.2307/2685660.

[52] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition; Dec. 2016. p. 770–8. https://doi.org/10.1109/CVPR.2016.90.

[53] Barabási AL, Albert R. Emergence of scaling in random networks. Sci (80-) 1999; 286(5439):509–12. https://doi.org/10.1126/science.286.5439.509.

[54] Taud H, Mas JF. Multilayer Perceptron (MLP). Geomatic approaches for modeling land change scenarios. Cham: Springer International Publishing; 2017. p. 451–5. https://doi.org/10.1007/978-3-319-60801-3_27.

[55] Watts DJ, Strogatz SH. Collective dynamics of 'small-world' networks. Nature 1998;393(6684):440–2. https://doi.org/10.1038/30918.

[56] Holme P, Kim BJ, Yoon CN, Han SK. Attack vulnerability of complex networks. Phys Rev - Stat Phys Plasmas Fluids Relat Interdiscip Top 2002;65(5):056109. https://doi.org/10.1103/PhysRevE.65.056109.

[57] Geisberger R, Sanders P, Schultes D. Better approximation of betweenness centrality. In: 2008 proceedings of the tenth workshop on algorithm engineering and experiments (ALENEX); 2008. p. 90–100.
[58] Wu M, Mou J, Dai B, Tan S, Lu X. Dismantling directed networks: a multi-temporal information field approach. Chaos Solit Fractals 2025;196(March):116404. https://doi.org/10.1016/j.chaos.2025.116404.
[59] Morone F, Makse HA. Influence maximization in complex networks through optimal percolation. Nature 2015;524(7563):65–8. https://doi.org/10.1038/nature14604.
[60] Zdeborová L, Zhang P, Zhou H. Fast and simple decycling and dismantling of networks. Sci Rep 2016;6(1):37954. https://doi.org/10.1038/srep37954.
[61] Bury TM, et al. Deep learning for early warning signals of tipping points. Proc Natl Acad Sci U S A 2021;118(39):e2106140118. https://doi.org/10.1073/pnas.2106140118.
[62] Shi D, et al. Local dominance unveils clusters in networks. Commun Phys 2024;7(1):170. https://doi.org/10.1038/s42005-024-01635-4.
[63] Li R, Yang R, Shi Y, Liu J, Chen B. Hierarchy of urban road networks from a scaling law perspective. Cities 2025;166(May):106215. https://doi.org/10.1016/j.cities.2025.106215.